

Examora — Project Analysis Report

1. Project Overview

Examora is a machine learning-driven system designed to automatically predict the difficulty level of exam questions (Easy, Medium, Hard) and provide pedagogical feedback for improvement. It combines classical NLP techniques with an experimental agentic AI assistant to support educators in creating and evaluating assessment content.

2. Problem Statement

Educational institutions often rely on manual evaluation of question difficulty, which can be subjective, inconsistent, and time-consuming. Examora addresses this challenge by:

- Automating difficulty prediction using textual and statistical features
 - Providing consistent and data-driven insights
 - Enhancing question quality through AI-generated recommendations
-

3. Objectives

- Predict exam question difficulty using machine learning
 - Provide actionable pedagogical feedback
 - Support both single and batch evaluation workflows
 - Visualize model performance for transparency
-

4. System Architecture

4.1 High-Level Workflow

1. User inputs question (UI)
 2. Preprocessing of text and metadata
 3. Feature engineering (TF-IDF + numeric features)
 4. Model prediction
 5. Output visualization and feedback
 6. Agentic AI generates recommendations
-

5. Tech Stack

Languages: Python 3.x

Frontend/UI: Streamlit

ML & Data: pandas, numpy, scikit-learn, scipy, joblib

NLP: NLTK, TF-IDF (scikit-learn)

Visualization: Plotly, matplotlib, seaborn

Agentic AI : LangChain, LangGraph, Groq API, HuggingFace, FAISS

Environment: python-dotenv

6. Project Structure

```
exam_question_analysis/
├── app.py
├── src/
│   ├── preprocessing.py
│   ├── feature_engineering.py
│   ├── train.py
│   ├── evaluate.py
│   └── agent.py
├── data/
│   └── exam_question_dataset_5000.csv
├── models/
│   ├── best_model.pkl
│   ├── tfidf_vectorizer.pkl
│   ├── scaler.pkl
│   ├── label_encoder.pkl
│   ├── results_summary.json
│   └── confusion_matrices/
├── notebooks/
├── report/
└── README.md
```

7. Core Features

7.1 Single Question Prediction

- Accepts question text and metadata
- Performs preprocessing and feature extraction

- Outputs predicted difficulty with confidence score
- Provides contextual tips

7.2 Batch Prediction

- Upload CSV file
- Processes multiple questions
- Generates predictions and visual analytics
- Allows result download

7.3 Model Dashboard

- Displays accuracy, F1 score, and evaluation metrics
- Shows confusion matrix visualizations

7.4 Agentic AI Assistant (Experimental)

- Uses LangGraph workflow
 - Retrieves pedagogy knowledge via FAISS
 - Performs web search
 - Generates structured JSON output:
 - reasoning
 - learning gaps
 - recommendations
-

8. Core Modules

preprocessing.py

- Text cleaning (stopwords, lemmatization)
- Topic encoding and one-hot encoding
- Label mapping

feature_engineering.py

- TF-IDF vectorization (max 300 features)
- Numeric feature scaling
- Sparse matrix construction
- Gaussian noise injection (training only)

train.py

- Defines and trains models:
 - Logistic Regression
 - Decision Tree

- Random Forest
- Saves best model and artifacts

evaluate.py

- Computes evaluation metrics
- Generates confusion matrix plots

agent.py

- Implements LangGraph-based workflow
 - Integrates RAG + web search + LLM
-

9. Code Flow

Prediction Flow

Input → Preprocessing → Feature Engineering → Model Prediction → Output Display

Training Flow

Dataset → Preprocessing → Feature Engineering → Model Training → Evaluation → Artifact Storage

Agent Flow

User Query → Retrieval (FAISS) → Web Search → LLM Generation → JSON Output

10. Setup & Installation

Step 1: Create Virtual Environment

```
python -m venv .venv  
source .venv/bin/activate
```

Step 2: Install Dependencies

```
pip install -r requirements.txt
```

Step 3: Configure Environment

Create a `.env` file:

```
GROQ_API_KEY="your_api_key"
```

Step 4: Train Model

```
python train.py
```

Step 5: Run Application

```
streamlit run app.py
```

11. Strengths

- Modular and clean architecture
 - User-friendly Streamlit interface
 - Reproducible ML pipeline
 - Multiple model comparison
 - Integrated evaluation metrics
 - Innovative agentic AI integration
-

12. Limitations

- Dependency on external APIs for AI assistant
 - No unit testing or CI/CD pipeline
 - Limited input validation
 - Not optimized for large-scale deployment
 - Minimal model interpretability
 - Potential failure in LLM JSON parsing
-

13. Future Enhancements

Engineering

- Docker-based deployment
- FastAPI backend for scalability
- CI/CD integration

Machine Learning

- Hyperparameter tuning

- Transformer-based embeddings
- SHAP/LIME interpretability

Agent Improvements

- Robust output parsing
- Offline LLM fallback
- Persistent knowledge base

UX & Security

- Input validation and error handling
 - Secure secret management
 - Rate limiting and logging
-

14. Conclusion

Examora is a well-structured and practical machine learning application that bridges classical NLP techniques with modern AI capabilities. It is suitable for academic use and demonstrates strong design principles. With improvements in scalability, reliability, and interpretability, it has the potential to evolve into a production-grade educational tool.

15. Key Files

- `app.py` — UI and entry point
- `preprocessing.py` — Data cleaning
- `feature_engineering.py` — Feature pipeline
- `train.py` — Model training
- `agent.py` — AI assistant
- `models/` — Saved artifacts